Tipos de Clústers y Tecnologías Asociadas

I. Introducción

1.1. Propósito y Alcance del Documento

El presente documento tiene como objetivo proporcionar una revisión técnica de las arquitecturas de clústers fundamentales en el ámbito de las Tecnologías de la Información:

- La Computación de Alto Rendimiento (HPC)
- La Computación de Alto Rendimiento de Tareas (HTC)
- La Alta Disponibilidad (HA)
- El Balanceo de Carga (LB)
- El Almacenamiento Distribuido.

Para cada tipología, se examinará su definición, criterios de aplicabilidad, tecnologías de implementación más utilizadas y los casos de uso más relevantes, detallando la justificación arquitectónica para cada aplicación.

Este análisis busca ofrecer una comprensión matizada de los compromisos inherentes a cada diseño de clúster, facilitando la toma de decisiones informadas en la planificación de infraestructuras críticas.

1.2. Definición de Clúster

En el contexto de las Tecnologías de la Información, un clúster se define como un conjunto de computadoras interconectadas, denominadas nodos, que trabajan conjuntamente como una única unidad cohesionada para lograr objetivos específicos.

La computación distribuida es el principio operativo que rige estos sistemas, permitiendo que la potencia de cálculo, la capacidad de almacenamiento o la disponibilidad de servicios se distribuyan entre múltiples componentes. Esta arquitectura distribuida es esencial para afrontar retos que superan las limitaciones de los sistemas informáticos monolíticos estándar, ya sea por la exigencia de velocidad, volumen o continuidad operativa.

II. Clústers de Computación de Alto Rendimiento (High Performance Computing - HPC)

2.1. Definición

La Computación de Alto Rendimiento (HPC) se define como el uso de arquitecturas de procesamiento paralelo que aprovechan la potencia combinada de múltiples sistemas informáticos de alta capacidad para resolver problemas complejos en ciencia, ingeniería y negocios.

El enfoque principal de HPC no es solo la capacidad de cálculo bruta, sino la minimización del *Time to Solution* (TTS). Esto requiere una optimización crítica de la comunicación inter-nodo, priorizando la **baja latencia**. Estos sistemas deben sobresalir en el procesamiento de grandes volúmenes de datos y en la ejecución de cálculos con alta precisión en coma flotante.

2.2. Aplicabilidad: Cuándo la Latencia y el Acoplamiento Fuerte son Críticos

Un clúster HPC aplica en aquellos escenarios donde las tareas son altamente interdependientes, es decir, existe un *fuerte acoplamiento*. La velocidad de cálculo y la comunicación extremadamente rápida entre nodos son indispensables, ya que la evolución del estado en un nodo depende de la información de sus vecinos. Cualquier retraso significativo (alta latencia) en la comunicación de mensajes paralizaría el clúster, ya que los nodos deben esperar la sincronización. Por lo tanto, HPC es la elección para problemas que no pueden dividirse en tareas completamente independientes.

2.3. Tecnologías de Implementación

Los sistemas HPC se basan en una combinación de componentes optimizados:

- **Hardware y Procesamiento:** Los nodos individuales utilizan CPU con múltiples núcleos y, de manera frecuente, aceleradores como Unidades de Procesamiento Gráfico (GPUs) o FPGAs para manejar los cálculos intensivos de punto flotante. La arquitectura puede ser homogénea (nodos de capacidad similar) o heterogénea.
- **Redes de Interconexión:** Se emplean interconexiones de alta velocidad y ultrabaja latencia (como InfiniBand u OmniPath) para facilitar la comunicación rápida entre nodos.
- **Entornos de Ejecución:** El estándar de programación *de facto* para la comunicación entre procesos fuertemente acoplados es **Message Passing Interface (MPI)**. MPI permite que los procesos compartan datos y se sincronicen, lo cual es fundamental para el paralelismo distribuido.

- **Gestión de Recursos (*Schedulers*):** Se utilizan sistemas como Slurm o PBS/Torque para gestionar la cola de trabajos y la asignación eficiente de recursos.

Un factor de cambio en la adopción de HPC es la creciente demanda de la computación en la nube. Tradicionalmente, mantener un clúster HPC actualizado con el hardware y software optimizados es un gasto continuo y significativo. Sin embargo, la HPC en la nube ofrece aprovisionamiento dinámico y escalado elástico, lo que permite a las empresas acceder a estos servicios sin la necesidad de construir sus propios centros de datos. Esta flexibilidad, que permite aumentar o reducir el uso de recursos sin esfuerzo, mejora la eficiencia y optimiza el uso de recursos. No obstante, los usuarios deben personalizar el sistema operativo y el software preinstalado para cumplir con los requisitos específicos de la carga de trabajo, lo que indica que, aunque la gestión del hardware se externaliza, la complejidad de la personalización del entorno HPC persiste.

2.4. Casos de Uso Frecuentes

2.4.1. Simulación Acoplada: Modelado Climático y Dinámica de Fluidos Computacional (CFD)

La necesidad de predecir el clima o simular flujos aerodinámicos (CFD) requiere dividir el espacio físico en una malla. La evolución de cada punto en la malla depende de los puntos vecinos. Esto implica que el proceso de cálculo de una región debe comunicarse constantemente y de forma síncrona con los procesos que manejan las regiones adyacentes.

Justificación del Clúster HPC: Se requiere la interconexión de baja latencia para que la comunicación mediante MPI sea efectiva. Si la latencia es alta, los nodos de cálculo pasarían la mayor parte del tiempo esperando la información de sus vecinos, anulando la ganancia del paralelismo. La capacidad combinada del clúster permite obtener resultados precisos rápidamente.

2.4.2. Investigación de Materiales a Escala Atómica (Cálculos de Primeros Principios - DFT)

Los cálculos basados en la Teoría del Funcional de la Densidad (DFT) buscan modelar el comportamiento cuántico de la materia para la investigación de nuevos materiales. Estos problemas son intrínsecamente intensivos, involucrando operaciones matriciales masivas y demandando una precisión extrema en las operaciones en coma flotante.

Justificación del Clúster HPC: La complejidad y el volumen de cálculo exigen la potencia agregada de múltiples sistemas interconectados. El procesamiento paralelo divide las tareas entre los núcleos y nodos, ejecutando numerosos cálculos simultáneamente, lo que reduce drásticamente el tiempo necesario para el descubrimiento científico.

III. Clústers de Alto Rendimiento de Tareas (High Throughput Computing - HTC)

3.1. Definición

La Computación de Alto Rendimiento de Tareas (HTC) se diferencia de HPC en su métrica de optimización. Mientras que HPC busca minimizar el tiempo de solución de una única tarea, HTC se enfoca en maximizar el **Throughput** (volumen de tareas completadas exitosamente por unidad de tiempo). HTC está diseñado para el manejo eficiente de grandes lotes de trabajos que son *débilmente acoplados*, es decir, que no requieren comunicación ni sincronización entre sí durante la ejecución.

3.2. Aplicabilidad: Cuándo el Volumen de Tareas y la Utilización Eficiente de Recursos son Prioridad

HTC aplica cuando un problema puede dividirse en miles o millones de tareas completamente independientes. El objetivo no es la velocidad de una tarea individual, sino la finalización del mayor número total de tareas en un periodo dado. Esto permite utilizar recursos computacionales de manera oportunista, incluyendo estaciones de trabajo o clústers heterogéneos y menos costosos que los utilizados en HPC, ya que la comunicación de baja latencia no es un factor limitante.

3.3. Tecnologías de Implementación

- **Arquitectura:** Los clústers HTC frecuentemente utilizan recursos heterogéneos, priorizando la utilización total del tiempo de CPU disponible.
- **Gestión de Cargas de Trabajo:** La suite de software **HTCondor (HTCSS)** es la tecnología líder, específicamente diseñada para automatizar y gestionar cargas de trabajo de alto *throughput*. HTCondor permite utilizar de manera efectiva todos los recursos informáticos disponibles, incluso aquellos dispersos.
- **Técnicas:** Para garantizar la robustez, se emplean técnicas de *checkpointing* y migración de trabajos, permitiendo que las tareas se detengan y reanuden en diferentes nodos de manera transparente para el usuario, manejando fallos o reasignación de recursos eficientemente.

La distinción entre HPC y HTC es la elección fundamental entre optimizar la **velocidad** (baja latencia) o el **volumen** (*throughput*) de trabajo. La latencia es el retraso en la comunicación, mientras que el *throughput* mide el volumen de paquetes de datos o tareas que llegan a destino exitosamente por unidad de tiempo. Si un problema es fuertemente acoplado (HPC), la latencia es crítica. Si es débilmente acoplado (HTC), el recurso caro (la red de ultrabaja

latencia) se vuelve innecesario, y la eficiencia se maximiza al tener un sistema de colas (scheduler) robusto para aprovechar la capacidad máxima de CPU, incluso si el inicio individual de cada trabajo tiene una alta latencia.

3.4. Casos de Uso Frecuentes

3.4.1. Simulaciones de Monte Carlo y Análisis de Grandes Conjuntos de Datos Científicos

En campos como la astrofísica o la física de partículas (ej. el Observatorio de Neutrinos IceCube), es necesario ejecutar la misma simulación o análisis miles de veces, variando parámetros de entrada para calcular probabilidades y reducir la incertidumbre estadística.

Justificación del Clúster HTC: Dado que cada simulación es una entidad independiente, un clúster HTC puede distribuir estos miles de trabajos en una "granja" de cómputo (a menudo gestionada por HTCondor), priorizando la utilización máxima del CPU para garantizar que el mayor número de resultados se obtenga por día. La necesidad de comunicación rápida (MPI) es inexistente, lo que hace de HTC la arquitectura idónea para este tipo de paralelismo masivo.

3.4.2. Descubrimiento de Fármacos y Cribado Virtual Masivo

El *cribado virtual* (*Virtual Screening*) implica la simulación computacional del acoplamiento (*docking*) de millones de posibles compuestos químicos a una proteína objetivo para identificar candidatos a fármacos.

Justificación del Clúster HTC: El proceso de simulación para un compuesto es independiente del resto. Los laboratorios farmacéuticos suelen utilizar HTC para este propósito, aprovechando la capacidad de procesamiento masivo para evaluar millones de interacciones moleculares en paralelo. Esto maximiza el *throughput* de pruebas moleculares, acelerando significativamente el descubrimiento de fármacos.

IV. Clústers de Alta Disponibilidad (High Availability - HA)

4.1. Definición

Los clústers de Alta Disponibilidad (HA) son arquitecturas redundantes cuyo propósito primordial es asegurar la continuidad operativa y mitigar el impacto negativo de un fallo del sistema primario, logrando esto mediante la **conmutación por error (*failover*)** automática y transparente. El diseño de HA busca alcanzar objetivos exigentes en términos de *Recovery Time Objective* (RTO) y *Recovery Point Objective* (RPO), idealmente manteniéndolos cerca de cero.

4.2. Aplicabilidad: Cuándo la Continuidad Operativa es Crítica

La HA es indispensable para recursos y servicios de misión crítica, donde el tiempo de inactividad genera un costo significativo (financiero, reputacional u operativo). Esto incluye sistemas de bases de datos transaccionales, servicios de autenticación corporativos, servidores de correo y sistemas de archivos esenciales que requieren acceso 24/7.

4.3. Tecnologías de Implementación

La implementación de HA se basa en la redundancia del sistema primario mediante un componente en modo de espera (*standby*) que puede activarse inmediatamente.

- **Suites de Clustering:** La gestión de la conmutación por error se articula mediante dos componentes principales:
    - **Corosync:** Proporciona la capa de pertenencia y comunicación del *clúster* (*Heartbeat*). Su función es vital para que los nodos puedan monitorear el estado de los demás.
    - **Pacemaker:** Es el gestor de recursos de alto nivel. Utiliza la información de Corosync para determinar el estado de salud de los servicios y automatizar el movimiento de los recursos (el *failover*) a un nodo superviviente en caso de fallo.

- **Sistemas de Bases de Datos:** Soluciones propietarias como Oracle Real Application Clusters (RAC) ofrecen *failover*, incluyendo la función TAF (Transparent Application Failover), que da soporte a la migración tras error de transacciones de solo lectura. Pacemaker y Corosync son también utilizados para configurar la HA en bases de datos empresariales como SQL Server en entornos Linux.

La alta disponibilidad se implementa típicamente en modos **Activo-Activo** o **Activo-Pasivo**. El modo Activo-Pasivo, aunque más simple para la gestión de la consistencia de datos, mantiene un nodo en espera, lo que resulta en una utilización de recursos más baja. Por el contrario, el modo Activo-Activo utiliza todos los nodos simultáneamente, lo que no solo proporciona redundancia, sino que también mejora el *throughput* y los tiempos de respuesta del clúster. La selección entre ambos modos representa un compromiso directo entre la eficiencia del recurso y la complejidad requerida para mantener la coherencia de datos en sistemas distribuidos.

4.4. Casos de Uso Frecuentes

4.4.1. Servidores de Bases de Datos Transaccionales (Ej. Oracle RAC, SQL Server)

Las bases de datos son el corazón de la mayoría de los sistemas empresariales y no pueden permitirse interrupciones. Un fallo puede resultar en una detención operativa costosa o la pérdida de datos si el *failover* no es instantáneo.

Justificación del Clúster HA: El uso de tecnologías como Oracle RAC o la configuración de SQL Server con Pacemaker/Corosync en Linux garantiza que, si un nodo de base de datos falla, el servicio se migra automáticamente a un nodo secundario. Aunque las transacciones de escritura pueden requerir ser reenviadas tras un error, la conmutación por error reduce drásticamente el tiempo de inactividad, manteniendo el servicio en línea.

4.4.2. Mantenimiento y Actualización de Sistemas de Archivos de Misión Crítica

Los sistemas de archivos distribuidos que soportan entornos de producción (ej. BeeGFS) requieren mantenimiento periódico, incluyendo actualizaciones de paquetes de Clustering (Pacemaker, Corosync). Un apagado completo del clúster para la actualización suele ser inaceptable en entornos 24/7.

Justificación del Clúster HA: La configuración de HA permite implementar una estrategia de actualización gradual. Este método implica poner cada nodo en modo de espera (standby) para mover los servicios a otro nodo y luego actualizar los paquetes. Este proceso requiere que el clúster esté sano y operativo antes de comenzar, utilizando el mecanismo de *failover* de manera controlada para asegurar que el servicio se mantenga operativo durante todo el periodo de mantenimiento.

V. Clústers de Balanceo de Carga (Load Balancing - LB)

5.1. Definición

El Balanceo de Carga (LB) es una metodología de infraestructura que distribuye el tráfico de red entrante (ej. solicitudes de clientes) a través de un conjunto de servidores *backend* (nodos del clúster) para mejorar la capacidad de respuesta, la *escalabilidad horizontal* y la disponibilidad de las aplicaciones.

5.2. Aplicabilidad: Cuándo se Requiere Manejar Grandes Volúmenes de Solicitudes y Evitar Cuellos de Botella

El LB aplica a cualquier servicio expuesto a alto tráfico externo, especialmente aplicaciones web, servicios de streaming o APIs, donde la demanda puede alcanzar miles de solicitudes por segundo. El balanceo de carga es esencial para prevenir que cualquier servidor individual se convierta en un cuello de botella y para agregar redundancia al sistema.

5.3. Tecnologías de Implementación

- **Software Balanceador:** Dos de las soluciones de software más comunes son **HAProxy** (altamente enfocado y especializado en balanceo de carga y proxying) y **NGINX/NGINX Plus**. NGINX ofrece una gama más amplia de funcionalidades, incluyendo el servidor web, que HAProxy no posee. NGINX Plus, en particular, ofrece capacidades de distribución dinámica avanzada, como la división de tráfico.

- **Algoritmos:** Los balanceadores utilizan algoritmos para la distribución:
    - **Algoritmos Estáticos:** Incluyen el Turno Rotativo Ponderado (*Weighted Round-Robin*), donde se asignan pesos a los servidores en función de su capacidad o prioridad, asegurando que los servidores más potentes reciban más tráfico.
    - **Algoritmos Dinámicos:** Consideran el estado actual de los servidores (ej. carga, número de conexiones activas) antes de enrutar el tráfico.

Los balanceadores de carga modernos han evolucionado para ser más que simples distribuidores de tráfico. Ofrecen funciones de seguridad, como la monitorización del tráfico y el bloqueo de contenido malicioso, además de la capacidad de redirigir automáticamente el tráfico de ataque a través de grupos de *firewalls* de red para una mayor protección. Esta funcionalidad ampliada posiciona al LB como un componente crítico de seguridad perimetral y gobernanza de tráfico.

5.4. Casos de Uso Frecuentes

5.4.1. Aplicaciones Web y APIs Masivas y Elásticas

Las plataformas que experimentan una demanda variable y masiva de usuarios (ej. comercio electrónico, redes sociales) necesitan gestionar miles de solicitudes simultáneas.

Justificación del Clúster LB: El balanceo de carga permite que estas aplicaciones escalen horizontalmente al agregar o eliminar servidores en el *backend* según la demanda. Al distribuir las solicitudes, se evita la sobrecarga de un servidor individual y se proporciona redundancia, ya que un fallo en un nodo no interrumpe el servicio, sino que el tráfico se redirige a los servidores restantes.

5.4.2. Canary Testing y Blue/Green Deployment

En entornos de integración y despliegue continuo (CI/CD), especialmente con arquitecturas de microservicios (como Kubernetes), es crucial minimizar el riesgo en la implementación de nuevas versiones.

Justificación del Clúster LB: Utilizando balanceadores avanzados como NGINX Plus y módulos como *ngx_http_split_clients_module*, es posible implementar la división dinámica de tráfico. Esto permite enrutar, por ejemplo, el 90% del tráfico de producción a un Clúster 1 (versión estable) y el 10% a un Clúster 2 (nueva versión para prueba). Esta técnica, conocida como Canary Testing (o Blue/Green Deployment en el caso de dos entornos iguales para la versión estable y la nueva), permite validar el nuevo código con tráfico real de manera controlada y, si surge un problema, cambiar instantáneamente la proporción a 100:0, eliminando el riesgo operativo.

VI. Clústers de Almacenamiento Distribuido

6.1. Definición

El Almacenamiento Distribuido es una arquitectura de *Software Defined Storage* (*SDS*) donde los datos se reparten y se replican entre múltiples nodos interconectados. El sistema resultante opera como una única entidad de almacenamiento lógico, proporcionando alta capacidad, redundancia y escalabilidad masiva. Su resiliencia se basa en la abstracción del hardware subyacente.

6.2. Aplicabilidad: Cuándo la Escalabilidad Masiva, la Resiliencia de Datos y el Manejo de Datos No Estructurados son Requisitos Clave

Estos clústers aplican cuando la gestión de grandes volúmenes de datos (a menudo en la escala de petabytes) es necesaria, o cuando se requiere una solución de almacenamiento que combine interfaces para el almacenamiento de bloques, objetos y archivos. Son fundamentales para arquitecturas de Big Data, análisis de datos y entornos de nube privada.

6.3. Tecnologías de Implementación

El mercado de SDS está dominado por soluciones de código abierto que garantizan resiliencia y escalabilidad:

- **Ceph:** Es una plataforma SDS unificada que ofrece almacenamiento de objetos (compatible con S3), almacenamiento de bloques (RBD) y sistemas de archivos distribuidos (CephFS). Es altamente valorado por su robustez y su arquitectura convergente.
- **GlusterFS:** Un sistema de archivos distribuido conocido por su simplicidad operativa y sus posibilidades para escalar a capacidades extremadamente altas (hasta 16 Exabytes).

La elección de la tecnología de almacenamiento distribuido debe basarse en el perfil de Entrada/Salida (I/O) de la aplicación. Aunque sistemas como GlusterFS ofrecen escalabilidad de capacidad masiva, el rendimiento de sus metadatos es notablemente bajo con un alto número de archivos. Esto implica que GlusterFS no es adecuado para cargas de trabajo transaccionales, como bases de datos, o para aplicaciones que dependen de un bloqueo estricto. Por lo tanto, se debe seleccionar la solución SDS basándose en si la carga de trabajo requiere alto *throughput* de grandes archivos (GlusterFS adecuado) o si es sensible a la latencia de metadatos y a las operaciones transaccionales (donde Ceph o soluciones de bloques dedicadas son más apropiadas).

6.4. Casos de Uso Frecuentes

6.4.1. Data Lakes para Big Data y Aprendizaje Automático (ML)

Los *Data Lakes* son repositorios que almacenan grandes volúmenes de datos en su formato nativo (incluyendo datos no estructurados) antes de que se defina un esquema. Esta flexibilidad es vital para los flujos de trabajo de ML y análisis avanzado.

Justificación del Clúster de Almacenamiento Distribuido: Estos clústers, particularmente aquellos que ofrecen almacenamiento de objetos compatible con S3, proporcionan la elasticidad y capacidad de ingesta necesarias para manejar datos no estructurados masivos. Permiten a las empresas de sectores como servicios financieros alimentar modelos de aprendizaje automático con datos de mercado en tiempo real para la gestión de riesgos.

6.4.2. Infraestructura de Nube Privada (IaaS)

Las plataformas de infraestructura como servicio (IaaS) o entornos de contenedores (Kubernetes) necesitan un *backend* de almacenamiento con características de alta disponibilidad y tolerancia a fallos para aprovisionar discos persistentes para máquinas virtuales o contenedores.

Justificación del Clúster de Almacenamiento Distribuido: Infraestructura para SDS como Ceph permite que una infraestructura de nube crezca dinámicamente. Al proporcionar simultáneamente almacenamiento de bloques y objetos, asegura que los discos virtuales y los *backups* de la infraestructura estén altamente disponibles y sean escalables. La tolerancia a fallos incorporada mediante replicación o codificación de borrado garantiza la persistencia de los datos, incluso ante fallos en el hardware.

VII. Tablas de Resumen Arquitectónico

La siguiente tabla resume las características distintivas y las métricas de optimización para cada tipología de clúster analizada.

Tabla VII.1: Resumen Comparativo de Arquitecturas de Clúster

Tipo de Clúster	Métrica de Optimización Principal	Foco Operacional	Dependencia de Tareas (Acoplamiento)	Tecnologías Clave
HPC	Latencia (Velocidad de Solución)	Cómputo intensivo, precisión	Fuerte Acoplamiento (Interdependientes)	MPI, InfiniBand, Slurm
HTC	Alto Throughput (Volumen de Tareas)	Tareas masivas, eficiencia de recursos	Débil Acoplamiento (Independientes)	HTCondor, Grid Engine
HA	Disponibilidad (RTO/RPO)	Resiliencia, Continuidad del negocio (Failover)	Alta (Protege recursos específicos)	Pacemaker, Corosync, Oracle RAC
LB	Escalabilidad (Distribución)	Rendimiento, Prevención de Cuellos de Botella	Variable (Distribuye solicitudes)	NGINX Plus, HAProxy, Algoritmos Dinámicos
Almacenamiento Distribuido	Capacidad/ Tolerancia a fallos	Almacenamiento masivo, gobierno de datos	N/A (Maneja datos distribuidos)	Ceph, GlusterFS, S3 (SDS)

La gestión de clústers de alta disponibilidad requiere comprender los modos operativos de redundancia:

Tabla VII.2: Modos Operacionales en Clústers de Alta Disponibilidad

Modo HA	Nodos Activos	Nodos Pasivos	Beneficio Principal	Inconvenientes
Activo-Pasivo	1	1 o más (Standby)	Simplicidad en la gestión de estado; *Failover* limpio	Utilización de recursos baja; el nodo pasivo está inactivo.
Activo-Activo	Todos	0	Alto *Throughput* y mejora de rendimiento general	Mayor complejidad en la gestión de concurrencia y consistencia de datos.

VIII. Glosario de Términos Clave

- **Latencia (*Latency*):** La demora en la comunicación de red, medida como el tiempo que tardan los datos en transferirse a través de la red. Es la métrica crítica que debe minimizarse en clústers HPC.
- **Throughput (Rendimiento):** El volumen promedio de datos o el número de tareas que pueden pasar exitosamente por un sistema durante un período específico. Es la métrica clave en HTC y LB.
- **Failover (Conmutación por Error):** La capacidad de un sistema redundante de cambiar de forma automática y transparente a un sistema de respaldo confiable (en espera/standby) cuando el componente primario falla, minimizando el impacto en el usuario.
- **Failback:** El proceso de devolver los servicios y la carga operativa del nodo secundario (que se activó durante el *failover*) al nodo primario original, una vez que este ha sido reparado y su funcionamiento ha sido verificado. Es el paso final para restaurar la configuración original y el uso óptimo de recursos en un clúster HA.
- **Escalabilidad Horizontal (*Horizontal Scaling*):** La capacidad de aumentar la capacidad o el rendimiento de un sistema agregando más nodos o máquinas a la arquitectura de clúster. Es la base de los clústers LB y de Almacenamiento Distribuido, permitiendo manejar volúmenes de tráfico crecientes.
- **Escalabilidad Vertical (*Vertical Scaling*):** La capacidad de aumentar la capacidad de un sistema agregando más recursos (como CPU, RAM o almacenamiento) a un único nodo o máquina existente. Está limitada por la capacidad máxima de *hardware* que podemos conectar a una máquina.
- **Pacemaker:** Un gestor de recursos de clúster de código abierto que supervisa el estado de los servicios críticos y automatiza el *failover* de recursos en clústers de Alta Disponibilidad.
- **Corosync:** La infraestructura de comunicación que proporciona la capa de pertenencia y el servicio de *heartbeat* esencial para que Pacemaker detecte y coordine el estado de los nodos HA.
- **MPI (Message Passing Interface):** Un estándar de programación utilizado en HPC que define las reglas para la comunicación paralela entre procesos. Es crucial para el rendimiento de tareas fuertemente acopladas.
- **RTO (Recovery Time Objective):** El tiempo máximo aceptable para restaurar las funciones empresariales después de una interrupción o desastre, un objetivo fundamental en la planificación de Alta Disponibilidad.
- **RPO (Recovery Point Objective):** La máxima cantidad de pérdida de datos (medida en tiempo) que es tolerable tras un evento de interrupción, un objetivo fundamental en la planificación de Alta Disponibilidad.
- **SDS (Software Defined Storage):** Una arquitectura de almacenamiento que abstrae la gestión de datos del hardware físico subyacente, permitiendo escalabilidad masiva, tolerancia a fallos y flexibilidad mediante software (ej. Ceph, GlusterFS).

- **Cribado Virtual (*Virtual Screening*):** Un caso de uso aplicado de HTC donde se emplean simulaciones computacionales masivamente paralelas para evaluar rápidamente grandes bibliotecas de compuestos químicos con fines de descubrimiento de fármacos.
- **Simulación de Monte Carlo:** Una técnica computacional que se basa en la repetición de muestreos aleatorios para obtener resultados numéricos. Se utiliza para modelar la probabilidad de diferentes resultados en un proceso que no puede predecirse fácilmente. En el contexto de HTC, implica la ejecución de la misma simulación o análisis miles de veces, variando parámetros de entrada para reducir la incertidumbre estadística y obtener proyecciones robustas.

IX. Conclusiones

El diseño eficiente de una infraestructura distribuida requiere una comprensión clara de la carga de trabajo primaria y la métrica de optimización más relevante. La elección entre las arquitecturas de clúster es un compromiso directo entre la velocidad, el volumen, la disponibilidad y la eficiencia de recursos.

1. **HPC y HTC:** La distinción más crítica reside en el acoplamiento de las tareas. Si las tareas dependen de la sincronización constante (fuerte acoplamiento), la baja latencia (HPC) es obligatoria. Si las tareas son independientes (débilmente acopladas), maximizar el *throughput* (HTC) mediante la utilización de recursos oportunistas es la solución más económica y eficiente.

2. **HA y LB:** La Alta Disponibilidad se centra en la resiliencia interna (*failover*) para la continuidad del negocio, mientras que el Balanceo de Carga se centra en la escalabilidad externa para manejar el volumen de tráfico. Los modernos balanceadores de carga también han adquirido un rol fundamental en la seguridad perimetral, actuando como guardianes del tráfico de aplicaciones.
3. **Almacenamiento Distribuido:** Al implementar SDS, la escalabilidad de capacidad no implica automáticamente un alto rendimiento para todas las cargas de trabajo. El perfil de I/O de la aplicación, especialmente la sensibilidad a la latencia de metadatos, debe guiar la selección entre soluciones como Ceph y GlusterFS, asegurando que la arquitectura de almacenamiento sea adecuada para las necesidades operacionales. El acceso a la computación avanzada a través de modelos de nube reduce las barreras de entrada para HPC, pero mantiene la necesidad de experiencia técnica para personalizar y optimizar los entornos de ejecución.

Enlaces de interés

1. ¿Qué es la informática de alto rendimiento (HPC)? - Supermicro https://www.supermicro.com/es/glossary/high-performance-computing
2. ¿Qué es un Clúster HPC? | Pure Storage https://www.purestorage.com/la/knowledge/what-is-an-hpc-cluster.html
3. Throughput vs Latency - Difference Between Computer Network Performances - AWS https://aws.amazon.com/compare/the-difference-between-throughput-and-latency/
4. ¿Qué es la computación de alto rendimiento (HPC)? - AWS https://aws.amazon.com/es/what-is/hpc/
5. ¿Qué es la computación de alto rendimiento (HPC) - Google Cloud https://cloud.google.com/discover/what-is-high-performance-computing?hl=es-419
6. HTCondor https://htcondor.org/
7. What is HTCondor? https://htcondor.org/description.html
8. What Is Failover? Definitions, Testing, & Importance in Systems | Druva https://www.druva.com/glossary/what-is-a-failover-definition-and-related-faqs
9. Actualice los paquetes Pacemaker y Corosync en un Clúster HA - NetApp https://docs.netapp.com/es-es/beegfs/administer/clusters-update-ha-services.html
10. Configuración de un Clúster de Pacemaker para grupos de disponibilidad de SQL Server https://learn.microsoft.com/es-es/sql/linux/sql-server-linux-availability-group-cluster-pacemaker?view=sql-server-ver17
11. Configuración de Oracle para alta disponibilidad - IBM https://www.ibm.com/docs/es/product-master/12.0.0?topic=database-oracle-setup-high-availability
12. ¿Qué es el equilibrio de carga? - Explicación del algoritmo de equilibrio de carga - AWS https://aws.amazon.com/es/what-is/load-balancing/
13. NGINX vs. HAProxy: Comparing Features and Use Cases | OpenLogic https://www.openlogic.com/blog/nginx-vs-haproxy
14. Controles de seguridad y equilibrio de carga dinámicos de múltiples clústers de Kubernetes A/B con NGINX Plus - F5 https://www.f5.com/es_es/company/blog/nginx/dynamic-a-b-kubernetes-multi-cluster-load-balancing-and-security-controls-with-nginx-plus.htmllbupstreamrequests/
15. Ceph vs GlusterFS? : r/sysadmin - Reddit https://www.reddit.com/r/sysadmin/comments/9onemk/ceph_vs_glusterfs/?tl=es-419
16. Data Lake vs. Data Warehouse: guía completa para las empresas modernas | InterSystems https://www.intersystems.com/es/recursos/data-lake-vs-data-warehouse/

17. ¿Qué es un data lake? | Google Cloud
https://cloud.google.com/learn/what-is-a-data-lake?hl=es-419
18. Sistemas de Archivos Distribuidos: Ceph y GlusterFS para Alta Disponibilidad y Escalabilidad - IT-CONSULTING
https://it-consulting.es/sistemas-de-archivos-distribuidos-ceph-y-glusterfs-para-alta-disponibilidad-y-escalabilidad/